

Platformer 5 - Raycast2D and Vector Math

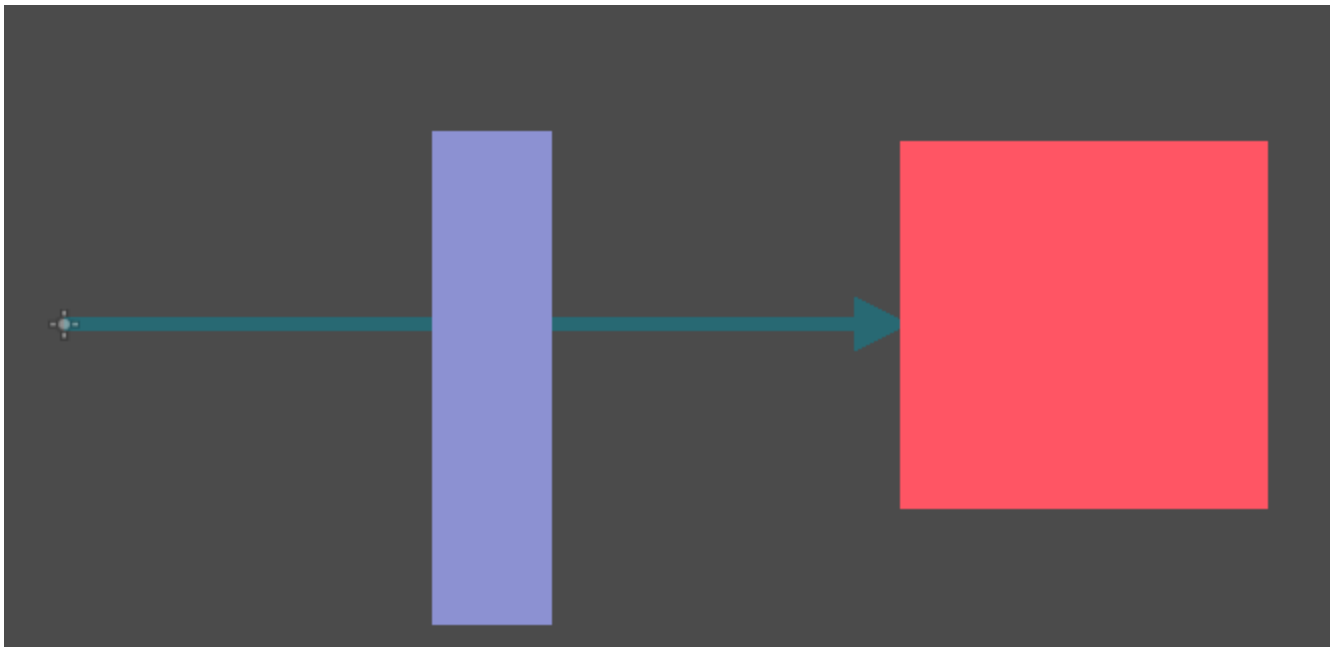
When creating enemies in a game we will often need to handle complex behaviors where the player is not *directly* controlling the entity. When this is happening, we need to get extra information about the world around the entity. Area2Ds are one tool that let us know about objects near an entity. However, we have another powerful tool called a Raycast2D that can be very useful when we need to respect level geometry (walls, floors, ceilings) such as if an enemy can see a player or if an enemy touches a wall.

Raycast2D

Raycast2Ds are a powerful tool for getting information about the game world. These act somewhat like an Area2D, but instead of a shape a Raycast2D is a single line, or ray, that extends out in a direction.



This ray can *collide* with PhysicsBodies, and then return back what PhysicsBody it collided with! This can be very useful when we want to respect floors and walls, which Area2Ds ignore. Importantly, a Raycast2D will only tell use the *first* object it collides with. In the scenario below, there is a wall and an object behind the wall. In this first example, even though the ray extends to the object behind the wall, *only* the wall will be returned as the ray hits the wall first:



Another key difference between Raycast2D and Area2D is that Raycast2D does *not* use signals. Instead, Raycasts update every frame and we use the Raycast2D's *methods()* to notify us when a collision has happened. We will use the following methods most often:

- `is_colliding()`
 - Returns true if the ray intersects an object
 - We'll use this method when we want to check if the ray is colliding with *anything*
- `get_collider()`
 - Returns the object collided with or null if no collision
 - We'll use this method when we want to check if the ray is colliding with a *specific* thing

`get_collider()` is a very powerful method(), but it requires us to be more careful when coding it than an area. First, we need to make sure the raycast is colliding with the right *kind* of `PhysicsBody`. We can do this with the **is** keyword in an if statement. Here's an example checking if the colliding body is a `CharacterBody2D`:

```
if raycast.get_collider() is CharacterBody2D:  
    pass
```

This will stop some nasty errors from being thrown. Next, we may want to check if the colliding object is in a group. We can do this in much the same way we did with groups! Here's an example:

```
if raycast.get_collider() is CharacterBody2D and  
raycast.get_collider().is_in_group("Player"):  
    pass
```

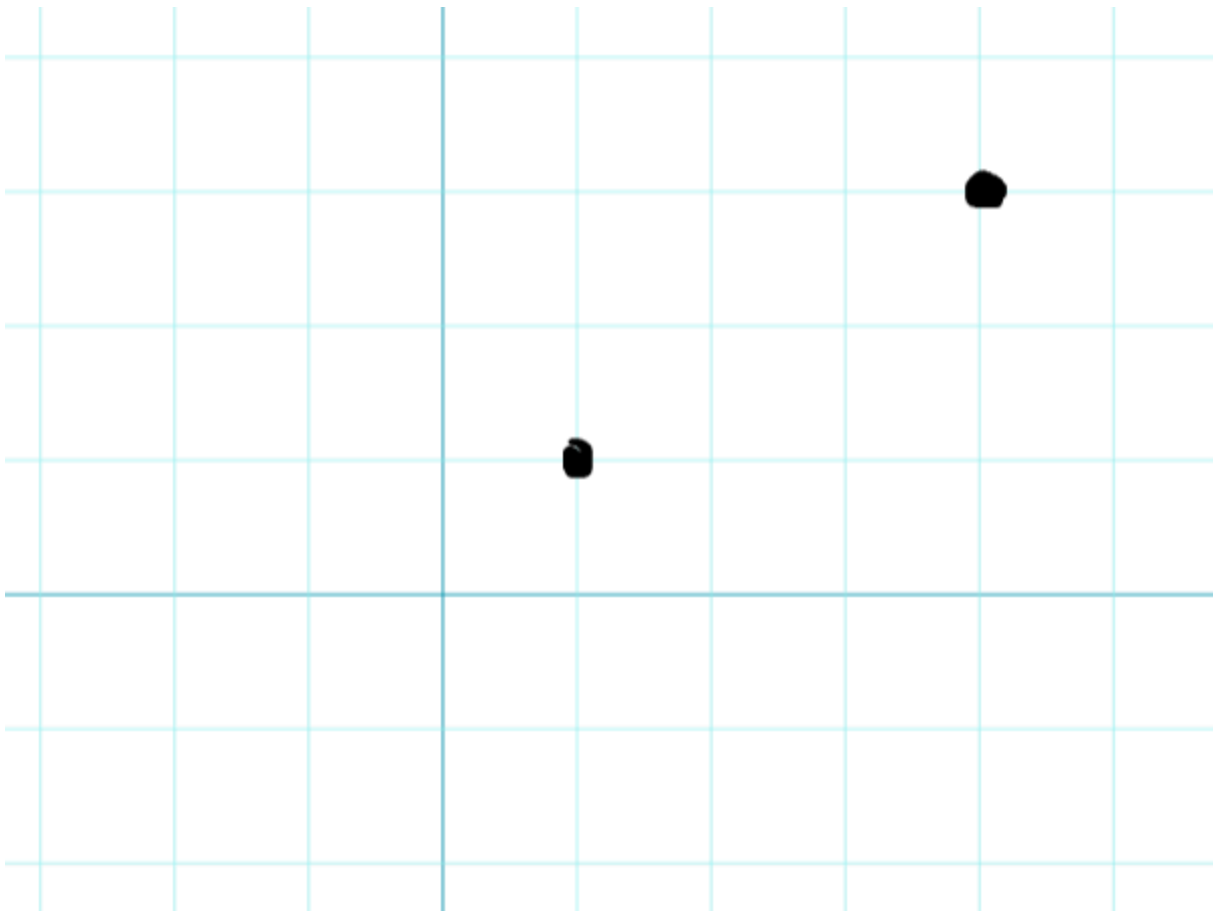
Then, we may want to do something to the object that we collided with. Here's an example where we `queue_free()` the colliding object:

```
if raycast.get_collider() is CharacterBody2D and  
raycast.get_collider().is_in_group("Player"):  
    raycast.get_collider().queue_free()
```

Vector Math

We've spoken some about Vectors, but let's dive into them a bit deeper. A `Vector2` is an x, y pair that can produce a *vector* in space. A *vector* is a line with *direction* and *magnitude* (or length). Vectors are very useful for when we want to *move* something at a fixed speed. A `CharacterBody2D`'s *velocity* property is just a *vector*, and the `CharacterBody2D` moves along that vector's direction the length of its' magnitude every frame. This means, if we can find the *vector* between any two points, we can move something in that direction!

For example, we may have a game where if an enemy sees our player they begin to move toward the player. Below is a graph, with one dot representing our player and the other our enemy:



Here, our player is at position (1, 1) and our enemy is at (4, 3). There are actually two possible *vectors* between these points. One pointing from the player to the enemy, and one pointing from

the enemy to the player. To find the vector between any two points we can use a very simple formula:

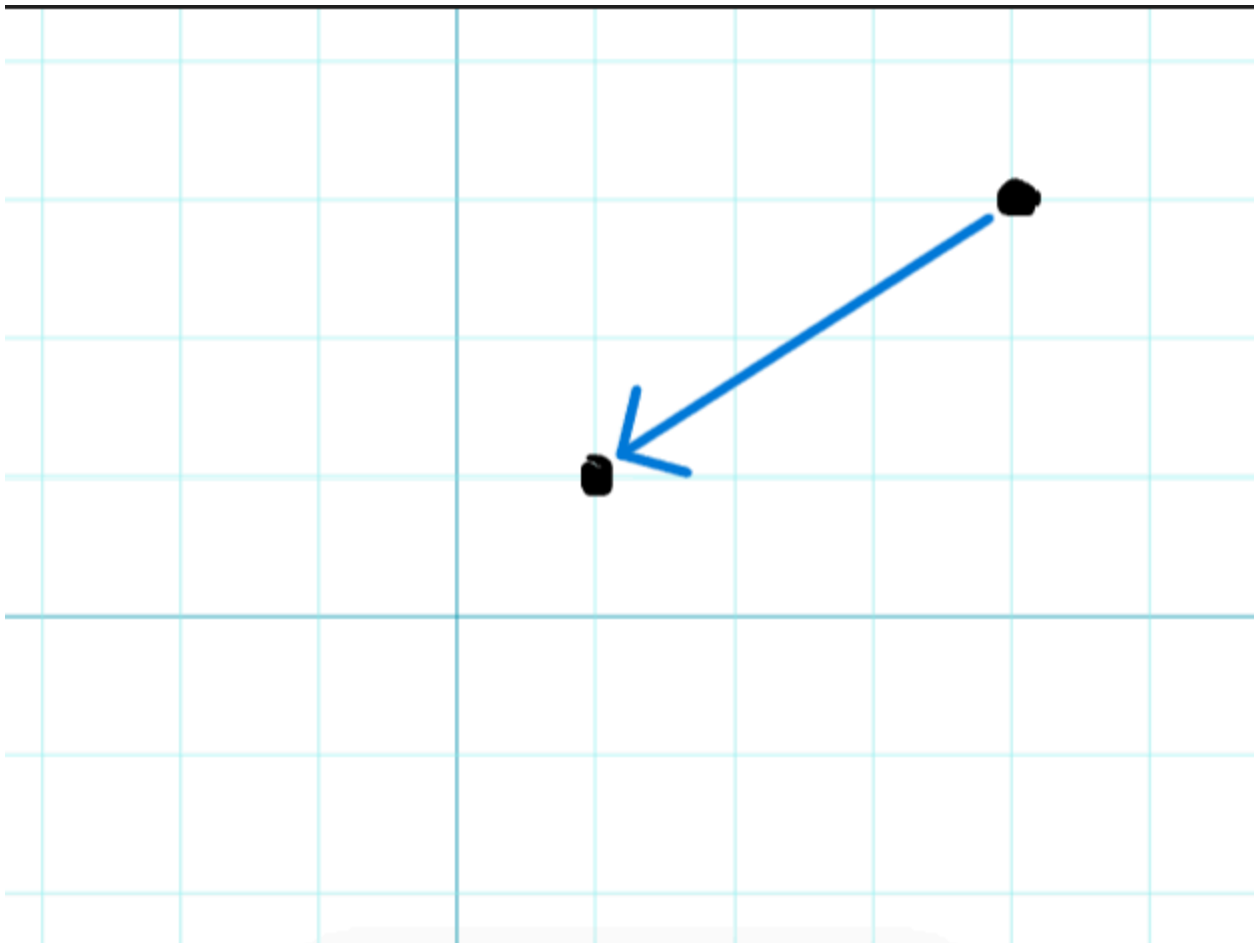
```
Vector2( Destinationx - Originx, Destinationy - Originy )
```

If we are finding the vector between the enemy and the player, the formula would look like this:

```
Vector2(1-4, 1-3)
```

Resulting in the following vector:

```
Vector2(-3, -2)
```



This is great! But what if our player was further away? We mentioned before that the speed at which a `CharacterBody2D` moves is based on the *magnitude* of the vector. If the player moves further away our formula will create a vector with a larger *magnitude* meaning the enemy will move faster when far away from the player and slower when its close. This isn't quite what we want. To fix this, we can first *normalize* the vector. This sets the vector to have a magnitude of 1. We do this by using `Vector2`'s `normalize()` method. Here's what that code might look like:

```
var new_vec = Vector2(player.x - enemy.x, player.y - enemy.y)
new_vec = new_vec.normalize()
```

```
enemy.velocity = new_vec
```

This will result in the enemy moving at the same speed regardless of the distance the player is from them. However, a magnitude of 1 will result in an extremely *slow* speed. Therefore, after normalizing the vector we then need to *scale* it. We can do this by multiplying the vector by a value.

```
var new_vec = Vector2(player.x - enemy.x, player.y - enemy.y)
new_vec = new_vec.normalize()
enemy.velocity = new_vec * 300
```

In this example, the normalized vector is used to find the *direction* to move. Then we *scale* the vector to have a consistent speed no matter the distance between the two entities. Often, we will want to use a variable instead of hard coding in a number for the scaling value. Here's an example of that:

```
var speed = 300
var new_vec = Vector2(player.x - enemy.x, player.y - enemy.y)
new_vec = new_vec.normalize()
enemy.velocity = new_vec * speed
```

Now it's your turn!

In your platformer project, perform the following tasks:

1. Add both of your enemies to a *enemy* group
2. In your player scene, add two Raycast2Ds at the bottom of your player facing down, placed near the edges of your player. If either ray intersects an object in the enemy group, call that enemy's `damage()` method
3. In one of your enemy scenes (The one that appears to be flying):
 1. Add a script to the root node, remove any code referring to falling or checking for inputs
 2. Add the following state functions:
 1. Idle
 1. The enemy moves back and forth slowly, changing directions every 2 seconds
 2. Attack
 1. The enemy moves directly towards the player
 3. Damage

1. The enemy is freed and the player has a negative y velocity force applied to it (the player bounces)
3. Add an Area2D (We'll use this to detect the player), this should have a circle CollisionShape2D that is quite large.
4. Change the current state when the following happens:
 1. When the player enters the Area2D change the state to attack
 2. Moves back to idle when the player exits the Area2D (Look for)
4. In your other enemy scene (The one that appears to be on the ground):
 1. Add a ray to each side of the enemy pointing away from it
 2. Add the following state functions:
 1. Move
 1. Moves the enemy in one direction, if either raycast hits a wall reverse its direction
 2. Damage
 1. 1st time the enemy's speed is doubled and the player has a negative y velocity force applied to it (the player bounces)
 2. 2nd time the enemy is freed and the player has a negative y velocity force applied to it (the player bounces)